CBOM Scanner

A command-line tool that scans a Python project and a website, lists every place cryptography is used, marks what quantum computers will break, and saves the result as a standard **CycloneDX 1.6 CBOM** plus an **HTML report**.

Final-year project GP059.

```
python scan.py ./sample-project --url example.com
```

```
  RED    RSA-2048              code        app/auth.py:5
  RED    X25519                tls         example.com:443
  YELLOW MD5                   code        app/legacy.py:6
  GREEN  AES-256               code        app/storage.py:12
  INFO   pyjwt                 dependency  requirements.txt:3
  ...
  20 crypto assets: 9 red, 4 yellow, 7 green  (+2 crypto libraries)
  Quantum readiness score: 35.0%

  CBOM:     cbom-output\cbom.json  (valid CycloneDX 1.6)
  Report:   cbom-output\report.html
```

Install

Python 3.11 or newer.

```
pip install -r requirements.txt
```

Usage

Command	What it does
`python scan.py ./myproject`	scan code, dependencies and certificate/key files
`python scan.py --url example.com`	check one website's TLS
`python scan.py ./myproject --url a.com --url b.com:8443`	both, several sites
`--out results`	output folder (default `cbom-output`)
`--skip deps` / `--skip certs` / `--skip code`	turn a scanner off

Outputs: `cbom.json` (CycloneDX 1.6, checked against the official schema on every run), `report.html` (summary bar, readiness score, filterable findings table) and `findings.json` (raw findings).

How it works

Four scanners return findings in one shared format; one step labels them and writes the outputs.

Part	File	Owner
Shared finding format, readiness score	cbom_scanner/finding.py	all
Red / yellow / green rules	cbom_scanner/rules.py	Samarth
Code scanner (Python `ast`)	cbom_scanner/code_scanner.py	Samarth
Rules file: which call means which algorithm	cbom_scanner/code_rules.py	Samarth
Dependency checker	cbom_scanner/dependency_scanner.py	Sarthak
Certificate / key scanner	cbom_scanner/cert_scanner.py	Sarthak
CycloneDX CBOM writer + schema validation	cbom_scanner/cbom_writer.py	Sarthak
TLS checker	cbom_scanner/tls_scanner.py	Sudhindra
HTML report + console output	cbom_scanner/report.py, templates/report.html.j2	Sudhindra
The single `scan` command	scan.py	Sudhindra

Every finding is a dictionary like:

```
{"source": "code", "file": "app/auth.py", "line": 5,
 "algorithm": "RSA", "param": 2048, "name": "RSA-2048",
 "primitive": "pke", "function": "keygen",
 "evidence": "rsa.generate_private_key(public_exponent=65537, key_size=2048)",
 "label": "red", "quantum_safe": False,
 "reason": "Public-key algorithm broken by Shor's algorithm on a quantum
computer"}
```

Code scanner. Parses each `.py` file with `ast` (so comments and strings are never findings), resolves imports (`from hashlib import md5 as m` → `hashlib.md5`), looks every call up in the rules file, and reads arguments for key sizes, curves and algorithm names. It follows simple variables (`ALGO = "sha1"; hashlib.new(ALGO)`, `key = os.urandom(32)`). Libraries covered: `hashlib`, `hmac`, `ssl`, `cryptography`, `pycryptodome(x)`, `rsa`, `PyJWT`/`python-jose`, `paramiko`, plus `pynacl`, `ecdsa`, `bcrypt` and `liboqs` (post-quantum). To support another library, add lines to `code_rules.py`; the scanner does not change.

Dependency checker. Reads `*requirements*.txt`, `pyproject.toml` (PEP 621 and Poetry) and `setup.py` (parsed, never executed), and lists known crypto libraries with versions. Libraries are labelled `info` and are not counted in the score, because a library is not an algorithm.

Certificate scanner. Opens `.pem .crt .cer .der .key .pub` files with `cryptography` and reports key algorithm and size, and for certificates also the signature algorithm.

TLS checker. Connects with Python's `ssl` module and records accepted TLS versions, cipher suite, key-exchange group and the server certificate. Python cannot see the TLS 1.3 key-exchange group, so the checker asks the `openssl` command for it when installed. That is what separates a classical `X25519` exchange (red) from a post-quantum hybrid `X25519MLKEM768` (green). Note: the group is the one negotiated with *your* OpenSSL; only OpenSSL 3.5+ can offer ML-KEM, and the report says so.

Labelling rules

Label	Meaning	Algorithms
red	broken by quantum computers (Shor)	RSA, DSA, DH, all elliptic-curve (ECDSA, ECDH, Ed25519, X25519), SSH via paramiko
yellow	weak today or weakened by quantum (Grover)	MD5, SHA-1, SHA-224, AES-128, DES, 3DES, RC4, Blowfish, TLS 1.0/1.1, unknown
green	quantum-safe	AES-192/256, SHA-256+, SHA-3, ChaCha20, HMAC-SHA-256+, ML-KEM, ML-DSA, SLH-DSA, TLS 1.2/1.3 (protocol version only)

Readiness score = green ÷ (red + yellow + green), as a percentage.

Evaluation (the three results tables)

```
python evaluation/evaluate.py        # table 1: 30 known-answer test files
python evaluation/real_projects.py   # table 2: 5 real open-source projects
(needs git + internet)
python evaluation/tls_checks.py      # table 3: TLS checker vs sslyze
python -m pytest -q                  # 13 unit tests
```

Results are written to evaluation/results/.

Table 1 – test files. The correct answer is written by hand next to each line as `# expect: MD5` (the scanner cannot see comments, so it cannot cheat). Certificates use expected_binary.json. 25 standard files and 5 *tricky* files (tc21–tc25) that show what the tool misses.

Group	Files	Expected	Correct	Missed	False alarms	Precision	Recall
Standard	25	53	53	0	0	100%	100%

Group	Files	Expected	Correct	Missed	False alarms	Precision	Recall
Tricky	5	9	2	7	1	66.7%	22.2%
All	30	62	55	7	1	98.2%	88.7%

Be honest about the 100%: we wrote both the test files and the rules, so the standard cases mostly show that the rules work as intended. Tables 2 and 3 are the independent checks.

Table 2 – real projects (main code; test-suite findings counted separately).
Fill in the hand check in `evaluation/results/spot_check.csv` (20 random findings).

Project	Findings	(in tests)	Red	Yellow	Green	Readiness
itsdangerous 2.2.0	3	0	0	3	0	0%
pyjwt 2.9.0	9	119	8	1	0	0%
python-rsa 4.9	4	8	4	0	0	0%
fabric 3.2.2	2	0	1	0	0	0%
flask-security 5.5.2	9	2	0	2	1	33.3%

Table 3 – TLS. On a local TLS 1.2 / RSA-2048 server, example.com and wikipedia.org, our checker and sslyze agree on accepted versions, certificate key, certificate signature and key-exchange group (12 of 12 checks).

Local test server: `docker compose -f evaluation/tls-testserver/docker-compose.yml up -d` (nginx, port 8443) or, without Docker, `python evaluation/tls-testserver/python_server.py` (port 8444).

What it cannot detect

- Algorithm names built at runtime (`"sha" + "1"`, f-strings), read from config files or environment variables
- Calls through `getattr()` or `importlib`, and algorithms selected via class attributes (e.g. pyjwt's `self.hash_alg`)
- Crypto *implemented* by hand (python-rsa's own math) or inside compiled libraries
- Crypto run outside Python (`subprocess.run(["openssl", ...])`)
- Variable scope: a function parameter that shadows a module name causes a false alarm (tc25)
- Other languages, transitive dependencies, certificate trust

Project layout

```
scan.py                    one command that runs everything
cbom_scanner/              the tool (≈1,700 lines, comments included)
sample-project/            demo project: RSA, MD5, SHA-1, SHA-256, AES-256,
HMAC, EC, a .pem
evaluation/
```

```
  testcases/                30 known-answer test files
  evaluate.py               table 1
  real_projects.py          table 2
  tls_checks.py             table 3
  tls-testserver/           nginx (Docker) and pure-Python TLS 1.2 test servers
  results/                  generated tables (.md and .csv), per-project CBOMs
and reports
tools/make_keys.py          regenerates all test certificates and keys
tests/                      pytest unit tests
```

All private keys in this repository are throwaway test keys made by `tools/make_keys.py`.